Graphics Programming

Vertex Buffer

GB. Tony Liang

OpenGL functions for Vertex Buffers (1/3)

- glGenBuffer(*bufferCount*)
  - Returns a set of variable vertex buffer references

- glBindBuffer(*bindTarget, bufferRef*)
  - Creates a buffer object referenced by *bufferRef* if no such object exisits
  - The buffer object is bound to *bindTarget*

OpenGL functions for Vertex Buffers (2/3)

- glBufferData(*bindTarget, bufferData, bufferUsage*)
  - Allocates storage for the buffer object
  - Currently bound to *bindTarget*
  - Upload *bufferData* into buffer
  - How data will be used is indicated by *bufferUsage*

- glGetAttribLocation(*programRef, variableName*)
  - Returns a value used to reference the attribute variable
  - The attribute variable is named as *variableName* and declared in program reference by *programRef*

OpenGL functions for Vertex Buffers (3/3)

- glVextexAttribPointer(*variableRef, size, baseType, normalize, stride, offset*)
  - Specifies that attribute variable *variableRef* will receive data from vertex buffer currently bound to GL_ARRAY_BUFFER, with element type *baseType*, number of components *size*.

- glEnableVertexAttribArray(*variableRef*)
  - Specified that *variableRef will be accessed and used during the rendering process.*

Graphics Programming

Draw Triangles

GB. Tony Liang

Vertex Array Objects (VAOs)

- Manage data involving attributes
- Can be bound and unbound as needed
- Stores information related to vertex buffer state, such as
  - Which buffers supply data to attribute variables in a program
  - How the data is arranged within each buffer

OpenGL functions for VAOs

- glGenVertexArrays(vaoCount)
  - Returns a set of available vertex array object references

- glBindVertexArray(*vaoRef*)
  - Binds a vertex array object reference by vaoRef
  - Unbinds any previously bound vertex array object

Graphics Programming

Passing Data Between Shaders

GB. Tony Liang

Type Qualifiers: *in* and *out*

- Vertex Shader
  - ***in*** – values will be supplied from a buffer
  - ***out*** – values will be passed to the fragment shader

- Fragment Shader
  - ***in*** – values will be supplied from the vertex shader
    - Note: interpolated during the rasterization stage
  - ***out*** – values written into buffer (color, depth)

Geometry Processing

- Determining the position of each vertex of the geometric shapes to be rendered
  - Implemented by *vertex shader*

Rasterization Stage

+ Determining which pixels correspond to the geometric shapes to be rendered

Pixel Processing

- Determining the color of each pixel in the rendered image
  - Using *fragment shader*

Type Qualifiers: *in* and *out*

- Vertex Shader
  - ***in*** – values will be supplied from a buffer
  - ***out*** – values will be passed to the fragment shader

- Fragment Shader
  - ***in*** – values will be supplied from the vertex shader
    - Note: interpolated during the rasterization stage
  - ***out*** – values written into buffer (color, depth)